

Day3-02 学习文档 v1.1 : Runtime Architecture —— Runtime Core Components

本文是《从零实现 Agent Runtime》学习阶段的 Day3-02 正式学习文档。

Day3-01 建立了 Runtime 的架构设计原则：Runtime 应该是 Orchestrator，而不是万能类。Day3-02 在这个基础上继续向下，把原则落到第一版核心组件设计上：Runtime、Conversation、Message、ContextBuilder、LLMClient、ToolRegistry。

与上一节的联系

Day3-01 解决的问题是：

Runtime 为什么应该这样设计？

它形成了几个关键判断：

- Runtime 是 Orchestrator，不是所有能力的集合。
- LLM 负责生成决策，Runtime 负责执行裁决。
- Tool 负责外部动作，但不是所有副作用都属于 Tool。
- Conversation 是事实存储，不应该知道 Summary、Context、LLM。
- 依赖方向必须保持单向。
- Runtime 能力应该通过组合扩展，而不是通过继承堆叠。

Day3-02 开始回答更具体的问题：

如果现在要写 mini-agent-runtime 的第一版，Runtime 内部到底需要哪些核心对象？

这一节暂时不讨论 Memory、Summary、Workflow、Streaming、Human Approval 等进阶能力，而是先把最小可运行 Runtime 的骨架搭起来。

目录

为什么先设计 Core Components？

第一版 Runtime 的核心对象

Runtime：流程调度器

Conversation：真实历史

Message：历史记录的最小单位

ContextBuilder：上下文编译器

LLMClient：模型调用边界

ToolRegistry：工具注册与查找

组件之间的依赖关系

一轮 Agent Loop 如何流动？

为什么暂时不把 Memory 和 Summary 放进核心？

当前阶段的架构判断

架构设计方法：从五问推导组件边界

下一节学习计划

写书 TODO

写书素材

1. 为什么先设计 Core Components ?

Agent Runtime 的第一版设计很容易走向两个极端。

第一个极端是过度简化：

```
async function run(input: string) {  
  const response = await openai.chat(input)  
  return response  
}
```

这个版本确实能调用 LLM，但它不是 Runtime。

它缺少：

- Conversation ;
- Message ;
- Context ;
- Tool Call ;
- Tool Result ;
- Loop Control ;
- 状态记录 ;
- 依赖边界。

第二个极端是过度膨胀：

```
class Runtime {  
  conversation  
  messages  
  prompt  
  llm  
  tools  
  memory  
  summary  
  workflow  
  streaming  
  approval  
  telemetry  
}
```

这个版本看起来很完整，但第一天就把所有未来能力都塞进 Runtime，会很快变成上帝类。

Day3-02 的目标是找到中间层：

不是只写一个 LLM wrapper，也不是设计一个万能 Runtime，而是建立最小但边界清晰的核心组件。

2. 第一版 Runtime 的核心对象

mini-agent-runtime 的第一版核心对象可以先收敛为：

```
Runtime
Conversation
Message
ContextBuilder
LLMClient
ToolRegistry
Tool
```

它们分别负责：

```
Runtime      = 控制 Agent Loop
Conversation = 保存真实历史
Message      = 表达一条对话事件
ContextBuilder = 从历史构建模型输入
LLMClient    = 屏蔽模型供应商差异
ToolRegistry = 注册、查找、执行工具
Tool         = 执行外部能力
```

如果用一句话概括：

Runtime 负责让这些对象协作，而不是替这些对象完成所有工作。

更具体地看：

```
User Input
|
v
Runtime
|
+-- Conversation.append(user message)
|
+-- ContextBuilder.build(conversation)
|
+-- LLMClient.chat(context)
|
+-- ToolRegistry.get(tool call)
|
+-- Tool.execute(args)
|
+-- Conversation.append(tool result)
```

这就是第一版 Runtime 的最小闭环。

3. Runtime：流程调度器

Runtime 的职责是控制流程。

它不应该负责：

- 直接拼 Prompt；
- 直接调用某个模型 SDK；
- 直接保存底层消息数组；
- 直接管理每个 Tool 的实现细节；
- 直接把所有能力逻辑写在自己内部。

Runtime 应该负责：

- 接收用户输入；
- 把用户输入写入 Conversation；

- 请求 ContextBuilder 构建上下文；
- 调用 LLMClient；
- 解析 LLM 返回；
- 判断是否存在 Tool Call；
- 通过 ToolRegistry 找到 Tool；
- 执行 Tool；
- 把 Tool Result 追加回 Conversation；
- 控制循环是否继续；
- 返回最终回答。

也就是说：

Runtime = Orchestration + Control

一个简化的 Runtime 形态可以是：

```
class Runtime {
  constructor(
    private conversation: Conversation,
    private contextBuilder: ContextBuilder,
    private llm: LLMClient,
    private tools: ToolRegistry
  ) {}

  async run(input: string): Promise<string> {
    this.conversation.append({
      role: "user",
      content: input,
    })

    while (true) {
      const context = this.contextBuilder.build(this.conversation)
      const response = await this.llm.chat(context)

      if (response.type === "final") {
        this.conversation.append({
          role: "assistant",
          content: response.content,
        })
        return response.content
      }

      const tool = this.tools.get(response.toolCall.name)
      const result = await tool.execute(response.toolCall.args)

      this.conversation.append({
        role: "tool",
        content: result,
        toolCallId: response.toolCall.id,
      })
    }
  }
}
```

这个代码不是最终实现，只是表达职责边界。

关键点是：

Runtime 控制循环，但具体能力交给组件。

4. Conversation：真实历史

Conversation 的职责非常窄：

保存真实发生过的对话历史。

它不应该知道：

- LLM 是什么；
- Tool 是什么；
- Context 是什么；
- Summary 是什么；
- Token 是什么；
- Runtime Loop 是什么。

它只需要提供少量方法：

```
class Conversation {
  private messages: Message[] = []

  append(message: Message) {
    this.messages.push(message)
  }

  all(): Message[] {
    return [...this.messages]
  }

  last(): Message | undefined {
    return this.messages.at(-1)
  }
}
```

Conversation 的核心价值不是复杂，而是可信。

它记录的是：

User 说了什么
Assistant 回了什么
LLM 请求了什么 Tool
Tool 返回了什么结果

这些都是事实。

Day2 已经形成过一个判断：

```
Conversation = Fact
Context      = View
Summary      = Cache
```

Day3-02 继续沿用这个判断。

Conversation 不负责为 LLM 优化输入，它只负责保存真实历史。

5. Message：历史记录的最小单位

如果 Conversation 是一组历史，那么 Message 就是一条历史记录。

Message 看起来简单，但它是 Runtime 后续扩展能力的基础。

第一版 Message 至少需要表达：

- 用户输入；
- Assistant 回复；
- Tool Call；
- Tool Result；
- 错误或中断状态。

一个简化的数据结构可以是：

```

type Message =
  | UserMessage
  | AssistantMessage
  | ToolCallMessage
  | ToolResultMessage
  | SystemMessage

type UserMessage = {
  role: "user"
  content: string
}

type AssistantMessage = {
  role: "assistant"
  content: string
}

type ToolCallMessage = {
  role: "assistant"
  toolCall: {
    id: string
    name: string
    args: unknown
  }
}

type ToolResultMessage = {
  role: "tool"
  toolCallId: string
  content: unknown
}

type SystemMessage = {
  role: "system"
  content: string
}

```

这里有一个重要边界：

Message 是 Runtime 内部的事实记录，不一定等同于某个 LLM SDK 的 message 格式。

例如 OpenAI、Anthropic、Gemini 对消息格式的要求可能不同。

如果把内部 Message 直接设计成某个供应商的格式，Runtime 会被模型 SDK 绑死。

更合理的设计是：

```

Internal Message
  |
  v
ContextBuilder / LLM Adapter
  |
  v
Provider-specific Message

```

这样 Runtime 内部模型保持稳定，供应商差异留在边界层处理。

6. ContextBuilder：上下文编译器

ContextBuilder 的职责是：

把 Conversation 中的真实历史，转换成当前这次 LLM 调用需要的 Context。

它不是事实存储。

它构建的是视图。

例如 Conversation 里可能有 1000 条消息，但本次 LLM 调用不一定全部传入。

ContextBuilder 可能会决定：

- 加入 system prompt；
- 选择最近 N 条消息；
- 插入工具说明；
- 插入当前任务状态；
- 插入 Summary；
- 插入 Memory；
- 按 token budget 裁剪上下文；
- 转换成目标 LLMClient 需要的输入格式。

所以更准确地说：

ContextBuilder 不是 Prompt Builder，而是 Context Compiler。

不要把它理解成：

```
Message[]  
|  
v  
String Prompt
```

这个理解太窄了。

Prompt 字符串只是最早期、最简单的一种 LLM 输入形式。随着 Runtime 能力增加，ContextBuilder 面对的输入会变成一组原始 Agent State：

```
Raw Agent State  
|  
+-- Conversation  
+-- Summary  
+-- Memory  
+-- Tool Schema  
+-- MCP Context  
+-- Policy  
+-- Runtime State  
|  
v  
Context Compiler  
|  
v  
LLM Context
```

未来多模态输入、RAG 结果、MCP 资源、Tool Schema、Token Budget、权限策略、工作流状态，都可能进入 Context。

因此 ContextBuilder 的价值不是“把消息拼成字符串”，而是：

在当前调用时刻，把 Agent 的相关状态编译成 LLM 可以消费的上下文。

第一版可以先保持简单：

```
class ContextBuilder {
  build(conversation: Conversation): Context {
    return {
      messages: conversation.all(),
    }
  }
}
```

但即使第一版很简单，也值得把它单独抽出来。

原因是 ContextBuilder 未来会变复杂。

如果第一版把上下文拼接逻辑写在 Runtime 里，后续加入 Summary、Memory、Tool Schema、Token Budget 时，Runtime 会不断膨胀。

所以更好的边界是：

```
Conversation 保存事实
ContextBuilder 构建视图
Runtime 使用视图调用 LLM
```

这也解释了为什么 Conversation 不应该知道 Summary。

Summary 不是 Conversation 的事实，而是 ContextBuilder 或 SummaryManager 在构建上下文时使用的辅助信息。

7. LLMClient：模型调用边界

LLMClient 的职责是隔离模型供应商。

Runtime 不应该直接依赖：

- OpenAI SDK；
- Anthropic SDK；
- Gemini SDK；
- 本地模型 SDK；
- 任意某个特定 API 的 response 格式。

Runtime 应该依赖一个稳定接口：

```
interface LLMClient {
  chat(context: Context): Promise<LLMResponse>
}
```

LLMResponse 也应该是 Runtime 能理解的内部格式：

```
type LLMResponse =
| {
  type: "final"
  content: string
}
| {
  type: "tool_call"
  toolCall: {
    id: string
    name: string
    args: unknown
  }
}
```



```
}
```

这样 Runtime 不关心底层模型是怎么返回的。

OpenAIClient 可以把 OpenAI response 转成内部 LLMResponse。

ClaudeClient 可以把 Claude response 转成内部 LLMResponse。

Runtime 只面对统一抽象。

这背后对应的是：

```
Runtime -> LLMClient Interface
OpenAIClient -> LLMClient
ClaudeClient -> LLMClient
```

也就是依赖接口，而不是依赖具体实现。

8. ToolRegistry：工具注册与查找

ToolRegistry 的职责是管理 Tool。

它应该负责：

- 注册工具；
- 按名称查找工具；
- 校验工具是否存在；
- 暴露工具描述给 ContextBuilder；
- 执行工具时提供统一入口。

一个简化设计可以是：

```
type Tool = {
  name: string
  description: string
  parameters: unknown
  execute(args: unknown): Promise<unknown>
}

class ToolRegistry {
  private tools = new Map<string, Tool>()

  register(tool: Tool) {
    this.tools.set(tool.name, tool)
  }

  get(name: string): Tool {
    const tool = this.tools.get(name)

    if (!tool) {
      throw new Error(`Tool not found: ${name}`)
    }

    return tool
  }

  list(): Tool[] {
    return [...this.tools.values()]
  }
}
```

Runtime 不应该自己维护一个散落的工具数组，也不应该在 run() 里写大量 if else：

```

if (toolName === "search") {
  return search(args)
}

if (toolName === "read_file") {
  return readFile(args)
}

```

这种设计很快会让 Runtime 和所有工具实现耦合。

更合理的是：

```
Runtime -> ToolRegistry -> Tool
```

Runtime 只知道自己需要执行某个工具调用。

ToolRegistry 负责找到工具。

Tool 负责真正执行。

9. 组件之间的依赖关系

Day3-02 的第一版依赖关系应该保持清晰：

```

Runtime
|
+-- Conversation
|
+-- ContextBuilder
|
+-- LLMClient
|
+-- ToolRegistry
    |
    +-- Tool

```

这表示：

- Runtime 可以依赖 Conversation；
- Runtime 可以依赖 ContextBuilder；
- Runtime 可以依赖 LLMClient；
- Runtime 可以依赖 ToolRegistry；
- ToolRegistry 可以依赖 Tool；
- Conversation 不依赖 Runtime；
- Message 不依赖 Runtime；
- Tool 不依赖 Runtime；
- LLMClient 不依赖 Runtime。

不应该出现：

```

Conversation -> Runtime
Tool -> Runtime
Message -> LLMClient
ContextBuilder -> Runtime

```

原因是这些反向依赖会让底层对象知道上层流程。

一旦底层对象开始知道上层流程，系统就会变得难以拆分、难以测试、难以复用。

Day3-01 的依赖方向原则在这里落地为：

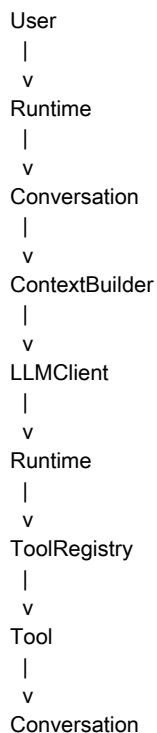
Runtime 依赖组件，组件不反向依赖 Runtime。

10. 一轮 Agent Loop 如何流动？

把这些组件连起来后，一轮 Agent Loop 可以这样理解：

1. User 输入任务
2. Runtime 接收 input
3. Runtime 追加 UserMessage 到 Conversation
4. Runtime 请求 ContextBuilder 构建 Context
5. ContextBuilder 读取 Conversation
6. ContextBuilder 返回 Context
7. Runtime 调用 LLMClient.chat(context)
8. LLMClient 返回 LLMResponse
9. Runtime 判断 response 类型
10. 如果是 final，追加 AssistantMessage 并结束
11. 如果是 tool_call，通过 ToolRegistry 查找 Tool
12. Runtime 执行 Tool
13. Runtime 追加 ToolResultMessage
14. 回到第 4 步继续循环

用更简化的图表示：



这里有一个重要点：

真正循环的是 Runtime。

LLM 不会自己循环。

Tool 不会自己循环。

Conversation 不会自己循环。

ContextBuilder 不会自己循环。

Runtime 才是控制整个流程继续或停止的对象。

11. 为什么暂时不把 Memory 和 Summary 放进核心？

Memory 和 Summary 都很重要，但不一定属于第一版 Core Components。

原因不是它们不重要，而是它们属于第二层能力。

第一版核心组件解决的是：

- 输入如何进入系统
- 历史如何保存
- 上下文如何构建
- 模型如何调用
- 工具如何执行
- 循环如何继续
- 结果如何返回

Memory 解决的是长期记忆。

Summary 解决的是上下文压缩。

它们应该在核心闭环成立之后再接入。

否则第一版设计会同时面对太多问题：

- 短期历史；
- 长期记忆；
- 摘要缓存；
- Token Budget；
- 检索；
- 写入策略；
- 过期策略；
- 冲突处理。

更稳妥的顺序是：

- Step 1: Runtime + Conversation + Message
- Step 2: ContextBuilder + LLMClient
- Step 3: ToolRegistry + Tool
- Step 4: SummaryManager
- Step 5: Memory
- Step 6: Workflow / Streaming / Approval

这也符合 Day3-01 的原则：

不要把未来能力提前塞进 Runtime，而是让核心结构能够通过组合扩展。

12. 当前阶段的架构判断

Day3-02 的核心结论是：

Agent Runtime 的第一版不需要很多组件，但每个组件都必须有清晰职责。

当前阶段可以形成以下判断：

- Runtime 是流程调度器，负责控制 Agent Loop。
- Conversation 是事实存储，只保存真实历史。
- Message 是历史记录的最小单位，不应该绑定某个模型供应商。

- ContextBuilder 负责从事实历史构建当前模型输入视图。
- LLMClient 负责隔离模型供应商差异。
- ToolRegistry 负责工具注册、查找和描述暴露。
- Tool 负责执行外部动作，并返回结构化结果。
- 组件之间应该保持单向依赖。
- Memory、Summary、Workflow、Streaming、Approval 都应该作为后续可组合能力接入。

也可以概括为：

```
Runtime controls the loop.
Conversation stores the facts.
ContextBuilder builds the view.
LLMClient calls the model.
ToolRegistry finds the tools.
Tool executes the action.
```

这就是 mini-agent-runtime 第一版架构的核心骨架。

13. 架构设计方法：从五问推导组件边界

Day3-02 不只是列出几个核心类，更重要的是学习如何从软件设计角度推导组件边界。

这节真正沉淀的方法论是：

```
Domain Model vs Domain Service
Ownership
State Boundary
Tell, Don't Ask
Runtime decides When, components decide How
```

这些方法会继续影响后续 Memory、Workflow、MCP、Streaming、Human Approval 的设计。

13.1 Domain Model 与 Domain Service

第一版 Runtime 组件可以分成两类。

第一类是 Domain Model：

```
Conversation
Message
```

Domain Model 表示系统中的事实。

Conversation 表示：

```
发生过什么
```

Message 表示：

```
某一个事实事件
```

它们的核心职责不是“提供能力”，而是用稳定的数据模型表达 Agent 运行过程中真实发生过的事情。

第二类是 Domain Service：

```
ContextBuilder
LLMClient
ToolRegistry
Tool
```

Domain Service 表示系统中的能力。

例如：

ContextBuilder = 构建上下文
LLMClient = 调用模型
ToolRegistry = 管理工具
Tool = 执行动作

这个分类很重要。

如果把 Conversation 当成 Service，它就很容易开始负责 Summary、Token、Context、LLM 调用。

如果把 ContextBuilder 当成 Model，它就会被误解成一个需要长期保存事实的对象。

更准确的边界是：

Domain Model = 表达事实
Domain Service = 完成能力
Runtime = 协调模型与服务

13.2 Ownership：谁拥有这个职责？

Day3-01 提出的第一个问题是：

Who owns it?

Day3-02 里每个组件都可以用这个问题检查。

例如 Message：

Who owns it?
Message 自己拥有一条事实事件的表达。

Who decides?
Runtime 决定何时创建 Message。

Who changes state?
Message 不应该被外部随意改写；如果需要合法性约束，应该由 Message 自己维护。

Who depends on whom?
Conversation -> Message。

Can this be composed?
可以。UserMessage、AssistantMessage、ToolCallMessage、ToolResultMessage 都是 Message 的组成形态。

再例如 Conversation：

Who owns it?
Conversation 拥有消息历史。

Who decides?
Runtime 决定什么时候追加一条新消息。

Who changes state?
Conversation 通过 append() 改变自己的内部状态。

Who depends on whom?
Runtime -> Conversation -> Message。

Can this be composed?
可以。后续可以组合 Snapshot、Persistence、Event Log，但 Conversation 不需要知道 Runtime。

再例如 ContextBuilder：

Who owns it?
ContextBuilder 拥有上下文构建规则。

Who decides?
Runtime 决定什么时候需要构建 Context。

Who changes state?

ContextBuilder 第一版最好保持无状态；即使未来引入缓存，也应该由它自己封装。

Who depends on whom?

Runtime -> ContextBuilder -> Conversation。

Can this be composed?

可以。未来可以组合 Summary、Memory、Tool Schema、Policy、Token Budget。

这些分析说明：核心组件不是凭感觉拆出来的，而是通过职责归属推导出来的。

13.3 状态修改原则：从修改对象到请求对象完成职责

今天最重要的认知升级之一，是从“直接修改对象”转向“请求对象完成自己的职责”。

错误方向是：

```
runtime.messages.push(message)
```

或者：

```
conversation.messages.push(message)
```

这是一种面向过程的写法。

它的问题是调用方绕过了拥有状态的对象，直接修改内部数据。

更好的方向是：

```
conversation.append(message)
```

这表示：

```
拥有状态的对象
|
v
封装状态变化
|
v
提供行为接口
```

Conversation 拥有 messages，所以 Conversation 应该决定如何追加消息。

Runtime 只决定“现在需要追加一条消息”，而不是直接操作 messages 数组。

这背后的原则是：

```
Who owns state,
Who changes state.
```

谁拥有状态，谁就应该封装状态变化。

否则后续要加入校验、审计、事件、持久化、快照、回滚时，会发现所有状态修改都散落在 Runtime 和工具函数里。

13.4 Tell, Don't Ask

这也可以用面向对象里的一个经典原则表达：

Tell, Don't Ask.

不要先把对象内部数据拿出来，然后在外部替它做决定。

不好的写法：

```
const messages = conversation.all()
messages.push(message)
conversation.replace(messages)
```

更好的写法：

```
conversation.append(message)
```

也就是说，不是问 Conversation：

把你的内部数据给我，我来改。

而是告诉 Conversation：

请追加这条 Message。

这能让职责边界更稳定。

Runtime 不需要知道 Conversation 内部用数组、链表、数据库、事件日志还是快照存储。

Runtime 只需要知道：

```
conversation.append(message)
```

这就是请求对象完成自己的职责。

13.5 Runtime 决定 When，组件决定 How

Runtime 是 Orchestrator，所以它拥有流程上的时机判断。

但 Runtime 不应该替每个组件实现具体细节。

可以总结为：

Runtime decides When.
Components decide How.

例如：

Runtime 决定什么时候追加消息。
Conversation 决定如何保存消息。

Runtime 决定什么时候构建上下文。
ContextBuilder 决定如何编译上下文。

Runtime 决定什么时候调用模型。
LLMClient 决定如何适配具体模型供应商。

Runtime 决定什么时候执行工具。
ToolRegistry 决定如何查找工具。
Tool 决定如何完成外部动作。

如果 Runtime 开始决定 How，它会变成万能类。

如果组件开始决定 When，它会反向控制 Runtime Loop。

所以边界应该保持：

When = Runtime
How = Component

这也是 Day3-02 对 Day3-01 五问方法论的具体应用。

13.6 用五问重新审视核心组件

把第一版核心组件放进五问里，可以得到一个更稳定的检查表：

Runtime
Who owns it? Agent Loop
Who decides? Runtime 决定流程继续、暂停、结束
Who changes state? Runtime 只改变属于自己的运行状态，不直接改别人的内部状态
Who depends? Runtime -> Components
Can compose? 可以组合 LLMClient、ToolRegistry、ContextBuilder、Conversation

Conversation

Who owns it? 真实历史
Who decides? Runtime 决定何时写入
Who changes state? Conversation.append()
Who depends? Conversation -> Message
Can compose? 可以组合持久化、快照、事件日志

Message
Who owns it? 单个事实事件
Who decides? Runtime 或组件产出事件，Runtime 负责写入历史
Who changes state? Message 应尽量不可变
Who depends? 底层数据模型，不依赖 Runtime
Can compose? User / Assistant / ToolCall / ToolResult / System

ContextBuilder
Who owns it? 上下文编译规则
Who decides? Runtime 决定何时构建
Who changes state? 尽量无状态，缓存由自己封装
Who depends? ContextBuilder -> Conversation / Tool Schema / Summary / Memory
Can compose? 可以组合 Token Budget、Policy、RAG、MCP

LLMClient
Who owns it? 模型供应商适配
Who decides? Runtime 决定何时调用
Who changes state? LLMClient 不应该修改 Conversation
Who depends? 具体 Client 依赖供应商 SDK
Can compose? OpenAIClient / ClaudeClient / GeminiClient

ToolRegistry
Who owns it? 工具集合
Who decides? Runtime 决定何时查找和执行
Who changes state? ToolRegistry.register()
Who depends? ToolRegistry -> Tool
Can compose? 本地工具、MCP 工具、远程工具

这套分析方式会继续复用到后续章节。

Memory、Workflow、MCP、Streaming、Human Approval 都不应该凭感觉加入 Runtime，而应该先问：

Who owns it?
Who decides?
Who changes state?
Who depends on whom?
Can this be composed?

14. 下一节学习计划

下一节继续留在 Chapter3 Runtime Architecture 内部：

Day03-03 Conversation 与 Message 数据模型

重点问题是：

Conversation 和 Message 应该如何建模，才能既表达真实历史，又不被某个 LLM SDK 格式绑死？

会继续拆解：

Message Role
Message Type
Tool Call Message
Tool Result Message
Internal Message vs Provider Message
Conversation Append
Conversation Snapshot

Day03-02 解决的是核心组件职责。

Day03-03 会把其中最关键的数据模型单独展开。

15. 写书 TODO

TODO 1：补充「最小 Runtime 核心组件」章节

建议位置：

```
Chapter3 Runtime Architecture
|
v
3.2 Runtime Core Components
```

章节结构可以是：

为什么 Runtime 不是一个 LLM Wrapper？

第一版 Runtime 需要哪些核心对象？

Runtime / Conversation / Message 的关系

ContextBuilder 为什么必须独立？

LLMClient 为什么应该是接口？

ToolRegistry 为什么不应该写进 Runtime？

第一版 Agent Loop 时序

TODO 2：增加一个「LLM SDK 格式污染内部模型」反例

反例：

```
type Message = OpenAI.ChatCompletionMessageParam
```

问题是：

- Runtime 内部数据模型被 OpenAI 绑定；
- 切换 Claude 或 Gemini 时需要大面积改动；
- Tool Call、Tool Result、Streaming 的表示会被供应商格式牵着走；
- 测试时也需要构造供应商格式。

正确方向：

```
Internal Message
|
v
Provider Adapter
|
v
OpenAI / Claude / Gemini Message
```

TODO 3：补充「Context 是视图，不是事实」的小节

可以把 Day2 的判断延续到 Day3：

```
Conversation = Fact
Context      = View
Summary      = Cache
```

然后解释：

- Conversation 永远保存真实历史；
- ContextBuilder 可以裁剪、重排、注入系统提示；

- Summary 可以作为 Context 的一部分，但不能替代 Conversation；
- Runtime 使用 Context 调用 LLM，但不能把 Context 当作完整历史。

TODO 4：为 Chapter12 实现阶段准备类骨架

未来实现 mini-agent-runtime 时，可以先实现：

```
src/runtime.ts
src/conversation.ts
src/message.ts
src/context-builder.ts
src/llm-client.ts
src/tool.ts
src/tool-registry.ts
```

每个文件先保持小而清晰，避免一开始就加入 Memory、Summary、Workflow。

TODO 5：补充「从软件设计方法推导 Runtime 组件」章节

Day03-02 后半部分可以专门沉淀为书稿中的设计方法小节：

```
Domain Model vs Domain Service
Ownership
State Boundary
Tell, Don't Ask
Runtime decides When, components decide How
```

这一节不只是解释“有哪些组件”，而是解释“为什么这些组件应该这样分”。

16. 写书素材

素材 1：Runtime 不是 LLM Wrapper

可以保留以下句子：

一个只会调用 LLM 的函数，不是 Runtime。Runtime 的价值不在于调用模型，而在于管理一次智能行为从输入、上下文、决策、工具执行到结果返回的完整生命周期。

素材 2：组件职责类比

可以用这个类比解释第一版组件：

```
Runtime      = 导演
Conversation = 场记
Message      = 每一条场记记录
ContextBuilder = 剪辑师
LLMClient    = 演员沟通接口
ToolRegistry = 道具间
Tool         = 具体道具或外部能力
```

需要注意的是，这个类比只帮助理解职责，不应该替代正式定义。

素材 3：第一版 Runtime 的判断标准

可以作为检查清单：

```
如果去掉 Runtime，谁来控制循环？
如果去掉 Conversation，历史存在哪里？
如果去掉 Message，事实如何表达？
如果去掉 ContextBuilder，Prompt 拼接会污染哪里？
如果去掉 LLMClient，供应商差异会泄漏到哪里？
如果去掉 ToolRegistry，工具查找会散落在哪里？
```

素材 4：适合作为章节结尾的观点

可以保留以下句子：

Runtime 的第一版不是越完整越好，而是越能保持职责边界越好。

好的核心组件设计，会让未来能力自然接入；坏的核心组件设计，会让未来能力只能继续堆进 Runtime。

Agent Runtime 的骨架不是 Prompt，而是组件之间的职责关系。

资料来源

- [ChatGPT 分享记录：Day03-02 Runtime Core Components](<https://chatgpt.com/share/6a55f3be-3668-83e8-808a-b2a1155ba7b4>)
- [ChatGPT 反馈记录：Day03-02 Runtime Core Components](<https://chatgpt.com/share/6a55fa5a-8474-83ee-8f3f-e1f3007bd8f0>)
- 项目现有 Day3 拆分计划与 Day03-01 学习文档。